

SQL SERVER | T-SQL | SSMS

E-Commerce Order Management & Sales Analytics Database

A relational database project covering design, analytics, transactions, triggers, indexing and performance analysis

6

Tables

60

Business Reqs

3

Views

5

Procedures

4

Triggers

Author: Kusuma

Microsoft SQL Server | SSMS | T-SQL | [GitHub](#)

1 Project Overview

This project is a relational **E-Commerce Order Management and Sales Analytics Database** built with Microsoft SQL Server and T-SQL. It manages customers, products, categories, orders, order items and payments, and delivers business insights through SQL-based analysis.

It demonstrates practical SQL skills: database design, constraints, JOINS, subqueries, CTEs, window functions, views, stored procedures, transactions, triggers, indexes and query performance analysis.

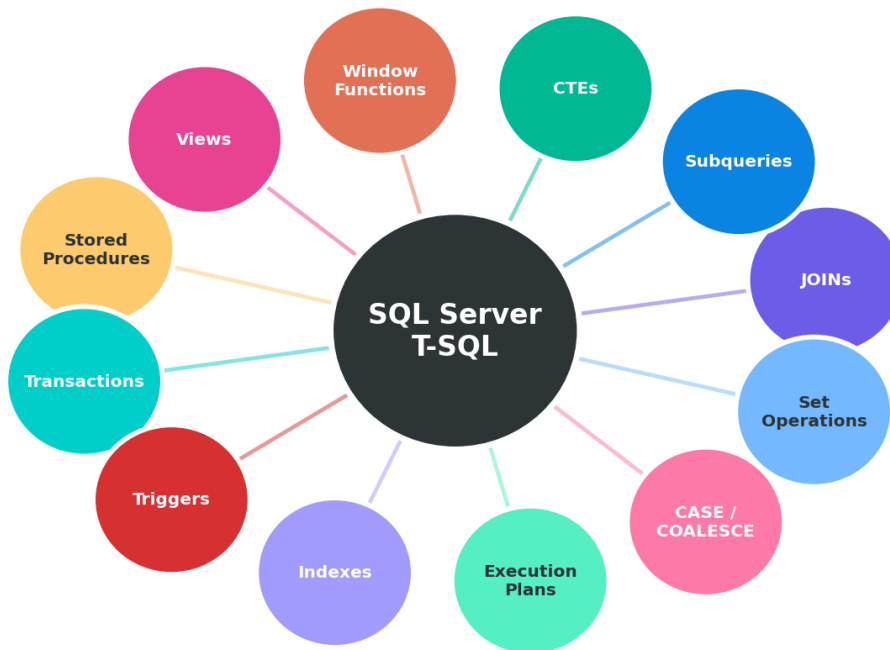


Figure 1 - SQL concepts demonstrated in the project

2 Project Objectives

- Design a relational e-commerce database.
- Manage customers, products, orders, order items and payments.
- Implement primary and foreign key relationships and data integrity constraints.
- Analyze customer purchasing behavior and product/category performance.
- Create reusable views and stored procedures.
- Demonstrate transactions, triggers, indexing and execution-plan analysis.

3 Technologies Used

Microsoft SQL Server

SSMS

T-SQL

GitHub

4 Database Structure

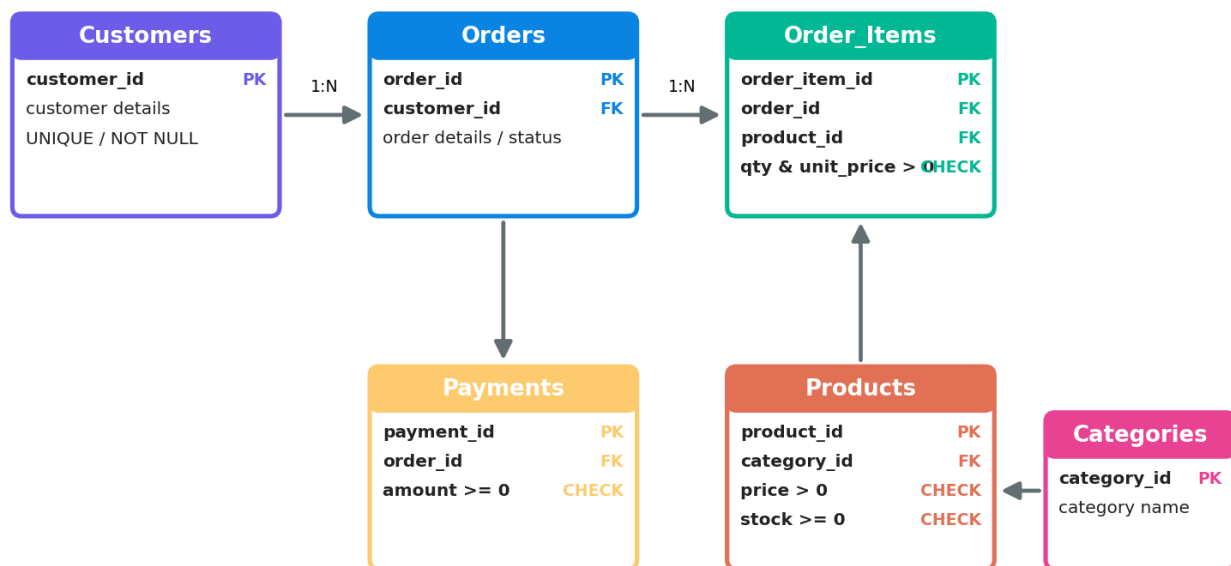


Figure 2 - Entity relationships (arrows point from parent to child table)

Relationships: Customers > Orders > Order_Items > Products > Categories. Payments are associated with Orders.

Customers	Customer information. PK: customer_id.
Categories	Product categories. PK: category_id.
Products	Product and inventory information. PK: product_id; category_id is an FK.
Orders	Customer orders. PK: order_id; customer_id is an FK.
Order_Items	Products within orders. PK: order_item_id; order_id and product_id are FKs.
Payments	Payment information. PK: payment_id; order_id is an FK.

5 Constraints & Data Integrity

PRIMARY KEY	FOREIGN KEY	UNIQUE
NOT NULL	DEFAULT	CHECK

Validation rules: positive product prices, non-negative stock, positive order-item quantities and unit prices, and non-negative payment amounts.

6 Business Analysis

The project contains **60 business requirements** covering practical e-commerce analysis:

Customer analysis & spending	Order & status analysis
Product & category performance	Revenue & sales analysis
Payment analysis	Low-stock & high-demand products
Customer activity	Average order value
Product rankings	Customer lifetime value

7 SQL Concepts Demonstrated

Database Design & Normalization	CRUD Operations	JOINS
Subqueries & Correlated	CTEs	Aggregate Functions
Window & Ranking Functions	CASE and COALESCE	Date & String Functions
Set Operations	Views	Stored Procedures
Transactions & ACID	Triggers	Indexes
Query Optimization	Execution Plans	SQL Server / T-SQL

8 Views

CustomerSpendingSummary	Customer order count, quantity and total spending.
ProductPerformance	Product quantity sold, revenue and category information.
CategoryRevenue	Revenue generated by each product category.

9 Stored Procedures

GetCustomerOrders	GetProductsByCategory
GetCustomerSpending	CheckCustomerSpending
GetCustomerOrderSummary	

These procedures demonstrate parameters, variables, aggregation, conditional logic, JOINS and COALESCE.

10 Transactions

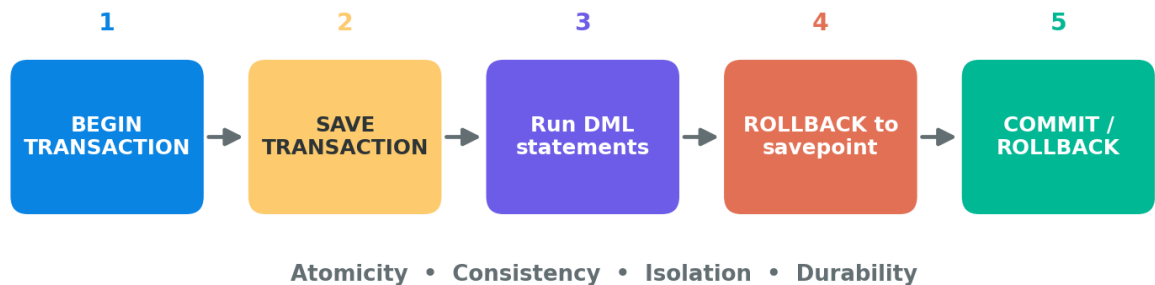


Figure 3 - Transaction control flow

The project demonstrates successful commits, complete rollbacks and savepoint-based partial rollback using BEGIN, COMMIT, ROLLBACK and SAVE TRANSACTION.

11 Triggers

trg_OrderInsertAudit	Records newly inserted orders.
trg_OrderStatusUpdate	Records order status changes.
trg_PreventNegativeStock	Prevents negative product stock.
trg_StockAudit	Records product stock changes.

12 Indexing & Performance

IX_orders_customer_id	Speeds up lookups of orders by customer.
IX_order_items_product_id_order_id	Supports product and order lookups on order items.
IX_payments_payment_status	Speeds up filtering payments by status.

SET STATISTICS IO	SET STATISTICS TIME	Actual Execution Plan
-------------------	---------------------	-----------------------

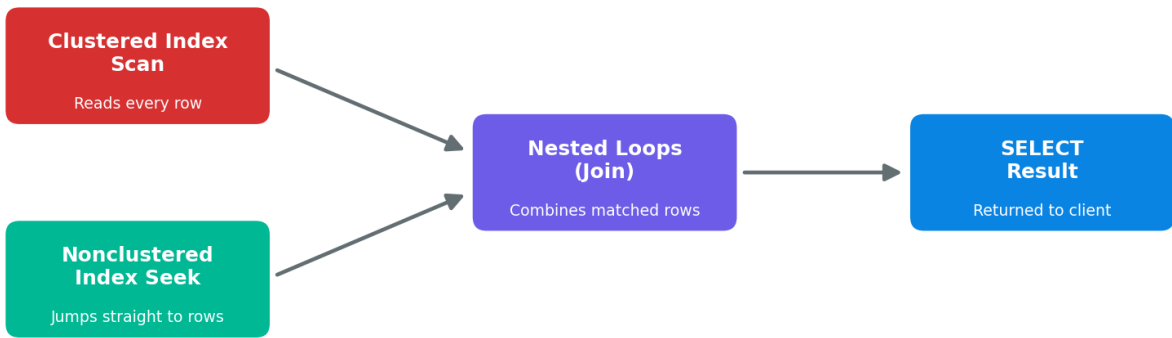


Figure 4 - Operators observed in execution-plan analysis

Execution-plan analysis demonstrated Clustered Index Scan, Nonclustered Index Seek and Nested Loops. Because the dataset is small, the tests demonstrate optimization concepts rather than production-scale benchmarks.

13 Key Project Outcomes

- Built a complete relational SQL Server project from design to analysis.
- Applied data integrity through keys and constraints.
- Used advanced SQL for business analysis.
- Created reusable views and parameterized stored procedures.
- Implemented transactions and audit triggers.
- Applied indexes and inspected SQL Server execution plans.
- Prepared the project for GitHub and portfolio presentation.

14 Project Files

ecommerce project.sql	Complete SQL implementation.
README.md	GitHub project documentation.
E-Commerce-SQL-Project-Report.pdf	Detailed project report.

15 How to Run

- 1 Open Microsoft SQL Server Management Studio (SSMS).
- 2 Create or select the **ecommerce_project** database.
- 3 Open **ecommerce project.sql**.
- 4 Execute the script in the appropriate order.
- 5 Verify tables, views, procedures, triggers, indexes and analysis queries.

16 Conclusion

This project demonstrates practical SQL Server and T-SQL skills across database design, data integrity, complex querying, business analytics, reusable database objects, transaction management, auditing, indexing and performance analysis.